

论高可用架构的设计与实现

⚠ 关于本文中的数字 本文的量化数据（可用率、响应时间、故障恢复时长等）均为**示例值**，用于展示论文应有的完整度与写法。**架构类数字**（表数量、接口数、配置值）已从项目源码核实，可直接使用；**业务规模类数字**（门店数、用户数、业务量、改善幅度）在项目资料中尚未确定，属示例值，背诵前请替换为你的**真实数据**。文末附有「待替换数字清单」，便于逐项核对。

试题

论高可用架构的设计与实现

高可用架构是指通过冗余设计、故障转移、容错处理、资源隔离等技术手段，使系统在部分组件发生故障时仍能持续对外提供服务，并将故障对业务的影响控制在可接受范围内。高可用架构设计需要明确系统的可用性目标，识别影响可用性的关键因素，并在成本与可用性之间做出合理权衡。

请围绕“高可用架构的设计与实现”论题，依次从以下三个方面进行论述：

- 简要叙述你参与设计和开发的高可用系统，以及你所担任的工作；
- 详细论述你在该系统中采取了哪些措施来保障系统的高可用性；
- 分析你在可用性保障过程中遇到的问题及解决措施，并说明如何验证系统的可用性。

摘要

本文以我参与建设的 H.I.S. BASE HOME 旅游业务平台为例，论述了高可用架构在该项目中的设计与实现。该平台服务日本 H.I.S. 集团营业本部与旅行者，业务具有强季节性——长假前预约量激增，平峰期迅速回落，对系统可用性提出了刚性要求。我在项目中担任系统架构师，负责可用性目标制定、故障隔离方案设计与可靠性保障机制建设。针对批量作业与在线业务争抢数据资源、消息推送存在重复发送风险、多实例部署下会话难以保持一致三大挑战，我从资源隔离、幂等重试、状态外置三个维度进行设计：为批量作业配置独立的读写数据源实现物理隔离；以执行状态的幂等校验与专用重试服务杜绝重复推送；以集中式会话存储实现 Web 层无状态化，支持任意水平扩容。系统上线后年可用率达到 99.95%，未发生因批量作业导致的在线服务降级，验证了方案的有效性。

正文

一、项目背景与可用性目标

BASE HOME 平台于 2024 年 3 月启动建设，2025 年 1 月上线运行，服务 H.I.S. 集团 300 余家营业门店与约 5000 名业务人员，日均处理预约约 3 万笔、消息与通知约 12 万条。我在项目中担任系统架构师，负责可用性目标制定、故障隔离方案设计以及可靠性保障机制的建设。

旅游业务对系统可用性有着刚性要求。预约窗口期一旦系统不可用，客户会转向其他渠道，流失的订单通常无法挽回；而通知与消息若漏发或重复发送，则会直接影响客户体验与品牌信誉。因此，在项目启动阶段，我与业务方及运维方共同定义了明确的可用性目标：年可用率不低于 99.9%；全部事务的 90 分位响应时间不超过 5 秒，

其中预约查询等核心场景进一步收紧至 2 秒以内；夜间批量作业必须在次日营业开始前完成；任何单一组件故障不得导致全站不可用。

在明确目标之后，我着手识别威胁可用性的关键因素。经过分析，我认为最大的威胁并非来自外部攻击或硬件故障，而是来自系统内部的三类问题：一是资源争抢，批量作业与在线业务共享数据库资源；二是重复处理，消息推送在失败重试时产生重复；三是状态耦合，多实例部署时用户的会话状态无法共享。下面分别论述我的设计。

二、高可用架构的总体设计

我的设计思路是围绕上述三类威胁逐层设防，形成资源隔离、故障域隔离、状态外置、数据层可靠性四层防护。

第一层是资源隔离。这是本项目中最关键的一层。平台的成本文件生成、外部预约导入、待办状态更新等批量作业需要从生产库大量读取并回写结果，而在线预约接口同时也在访问同一数据库。若两者共享连接池，夜间批处理峰值会直接推高在线接口的响应时间，严重时甚至导致连接耗尽。为此，我为批处理配置了独立的读写数据源：读取使用只读账号，写入使用写账号，并通过代码包路径隔离两套数据访问接口，使读写请求在物理层面彻底分离。同时，我把事务隔离级别设置为读已提交，减少长事务持有锁的时间；把批量游标大小限制在 200 条，防止海量结果集一次性载入导致内存溢出。这一层的设计牺牲了配置的简洁性，换来了在线业务的稳定，我认为这个交换完全值得。

第二层是故障域隔离。我把系统按部署单元划分为 Web、Service、Batch、FaaS 四类，使任一单元的故障不会扩散。其中批处理模块不启用 Web 容器，与在线业务没有进程级耦合；同步链路则按业务域拆分为十四个独立的函数，每个函数拥有独立镜像与独立部署单元，某个业务域的处理异常不会波及其他业务域。这一设计增加了运维单元的数量与管理成本，但把“一个模块崩溃导致全站不可用”的风险降为“单个业务域受影响”。

第三层是状态外置。Web 层多实例部署后，若会话存储在各个实例的本地内存中，负载均衡会把同一用户的请求分发到不同实例，导致用户被反复踢下线。我引入了集中式会话管理方案，把会话数据统一存放于云上无服务器形态的键值存储中，会话超时设为 28800 秒。这一取值是与业务方共同确定的：营业人员一个工作日的连续操作不应因会话过期而中断，8 小时恰好覆盖营业时段。会话外置后，Web 实例完全无状态，可以任意水平扩容与重启而不影响在线用户。

第四层是数据层可靠性。具体措施包括：对需要并发控制的实体采用乐观锁配合重试，冲突时以固定次数与固定间隔重试；把护照、文档等文件实体从数据库中剥离到对象存储，降低数据库体积与备份恢复时长；为集团主数据建立本地只读副本，使外部平台的故障不影响业务查询。

三、关键技术的实现

幂等控制，杜绝重复推送。消息与邮件推送是本平台最典型的可靠性敏感点，一旦重复发送，对客户而言就是直接的体验事故。我设计了三层防护：第一层是**执行状态幂等校验**，以专门的作业管理表记录每次作业的执行状态，同一作业处于“处理中”时重复触发会被直接拒绝——这一机制最初是为防范容器调度器的多重触发而设，后来成为全部作业的统一防线；第二层是把重试能力独立为专用的重试服务，与业务逻辑彻底解耦，最大尝试 4 次、间隔 1 秒，使重试策略可以独立调整而无需改动业务流程；第三层是投递状态跟踪表，记录每一次投递的结果（未投递 / 已投递），使异常场景下能够精确定位漏发与重发的范围。此外，我把事务边界拆分为独立的服务类，把事务粒度控制在必要范围内，避免大事务回滚造成连带重放。

可观测性建设。高可用的前提是“故障能被及时发现”。批处理属于短生命周期的进程，作业结束后进程即退出，常规的进程监控无法采集其运行状态。为此我引入了指标推送机制，由作业主动把执行指标推送到 Prometheus 的 PushGateway，并把关闭动作设为“进程退出前投递”，确保最后一批指标不会随进程退出而丢失。函数侧则借

助运行时框架输出结构化日志与链路追踪信息。这样一来，无论是批量作业失败还是同步链路异常，运维人员都能在监控面板上第一时间发现。

作业的可重跑与可复现。批处理的重跑能力是可靠性的重要保障。我把执行基准日、取得范围、导入类型等关键参数定义为作业参数显式传入，而不依赖系统时钟取当前日期。这一设计带来两个收益：一是补数时可以指定历史日期重跑，结果与当日运行完全一致；二是测试时可以固定参数，使结果可预期。对于跨国部署而言，这一点尤为重要——服务日本本部而容器运行于协调世界时，若依赖系统时钟判断“今天”，跨时区下极易取到错误的日期边界。

四、实施中遇到的问题及解决措施

问题一：批量作业峰值冲击在线服务。系统上线前的压测中，我们发现夜间批量作业执行期间，在线预约查询接口的响应时间由 300 毫秒劣化至 2.5 秒，已超出可用性目标。经分析，根源是两者共享数据库连接池，批量扫描占用了大量连接。我在上文的资源隔离方案正是针对此问题设计的——通过对批处理配置独立的读写数据源并按包路径隔离数据访问接口，使两者在物理层面分离。改造后再次压测，批量作业执行期间在线接口的响应时间稳定在 320 毫秒以内，批量作业自身耗时反而下降约 30%，实现了双赢。

问题二：消息重复推送。上线初期，我们收到客户反馈称同一封通知邮件收到了两次。排查发现，是邮件网关超时后系统自动重试，而首次请求实际上已经成功，重试造成了重复发送。这一事件的本质是分布式环境下的幂等性问题，而非简单的异常处理问题。我的解决方案即上文第三节所述的执行状态幂等校验、专用重试服务与投递状态跟踪三层防护。改造后，重复发送率由约 0.3% 降至 0.01% 以下。

问题三：多实例部署下的会话丢失。平台上线后随业务量增长需要横向扩容 Web 实例，但扩容后用户频繁被强制重新登录。原因是会话数据存储在每个实例的本地内存中，请求被负载均衡分发到其他实例时无法识别会话。我通过引入集中式会话存储解决该问题，并额外获得了一项安全收益：会话数据集中在服务端，可以强制失效任意用户的会话，从而实现“管理员踢出用户”“用户改密后全端下线”这类安全能力。

关于可用性的验证方式。我们采取了三层验证：一是在开发阶段利用容器化技术与本地云模拟工具搭建与云端一致的依赖环境，对读写分离、幂等控制等关键机制编写集成测试；二是在上线前进行压力测试，重点验证批量作业与在线业务并发执行时的相互影响；三是在运行期通过监控指标持续观测，把可用性目标拆解为可量化的监控项并设置告警阈值。

五、总结与展望

平台上线至今已稳定运行约 8 个月，年可用率达到 99.95%，超过了既定目标；批量作业均在次日营业开始前完成，未发生因批量作业导致的在线服务降级；消息与通知的重复发送率控制在 0.01% 以下。

目前仍存在两点不足：一是跨存储的最终一致性链路尚无统一的延迟监控，异常时的排查路径偏长；二是无服务器计算在低频函数上存在冷启动延迟，尚未引入预置并发配置。此外，需要客观说明的是，本平台目前采用的可靠性手段以资源隔离、故障域隔离、状态外置与幂等重试为主，尚未引入熔断与限流机制。在流量突增时，系统缺乏主动的自我保护能力，这是我们后续需要补齐的能力。

后续计划引入一致性延迟监控、预置并发配置以及服务端的熔断限流能力，并定期开展故障演练，验证在真实故障场景下的恢复能力。

通过本项目，我认识到高可用并不是靠某一项技术实现的，而是由一系列设计共同构成的体系：用资源隔离消除内部争抢，用故障域隔离限制故障扩散，用状态外置支撑水平扩容，用幂等与重试应对不可避免的失败。更重要的是，高可用必须有明确的目标与验证手段——没有量化目标的“高可用”只是一句口号，而无法验证的高可用则无法证明其存在。

写作提示（不计入论文字数，背诵时请忽略本节）

待替换数字清单

位置	示例值	请替换为
门店数 / 业务人员数	300 余家 / 约 5000 名	真实数量
日均预约量 / 消息量	约 3 万笔 / 约 12 万条	真实数量
年可用率目标 / 实际	99.9% / 99.95%	真实指标
响应时间目标	90 分位 5 秒以内（核心场景 2 秒）	✓ 已核实
压测劣化情况	300ms → 2.5s	真实压测数据
改造后响应时间	320 毫秒以内	真实压测数据
批量作业耗时下降	约 30%	真实数据
重复发送率	0.3% → 0.01%	真实数据
会话超时	28800 秒（8 小时）	✓ 已核实
重试策略	最大 4 次 / 间隔 1 秒	✓ 已核实
批量游标	200 条	✓ 已核实

本文字数

摘要约 330 字，正文约 2820 字，**合计约 3150 字。**

可复用要点

- **开头必写「可用性目标」**——这是高可用类论文的标准起手式，没有目标的论文会被判定为空谈
- **「高可用最大的威胁来自内部资源争抢而非外部攻击」** 是一个有洞察力的判断，能显著提升论文层次
- **结尾主动写出「尚未引入熔断限流」** 这一不足，既真实又体现架构判断力，比硬编造用了熔断组件安全得多
- 若题目问"如何验证可用性"，直接引用第四节末段的三层验证方式